

EXPERIMENT 13: ALPHA-BETA PRUNING ALGORITHM FOR GAMING

AIM

To implement the **Alpha-Beta Pruning algorithm** in Python to optimize the Minimax algorithm by eliminating unnecessary branches.

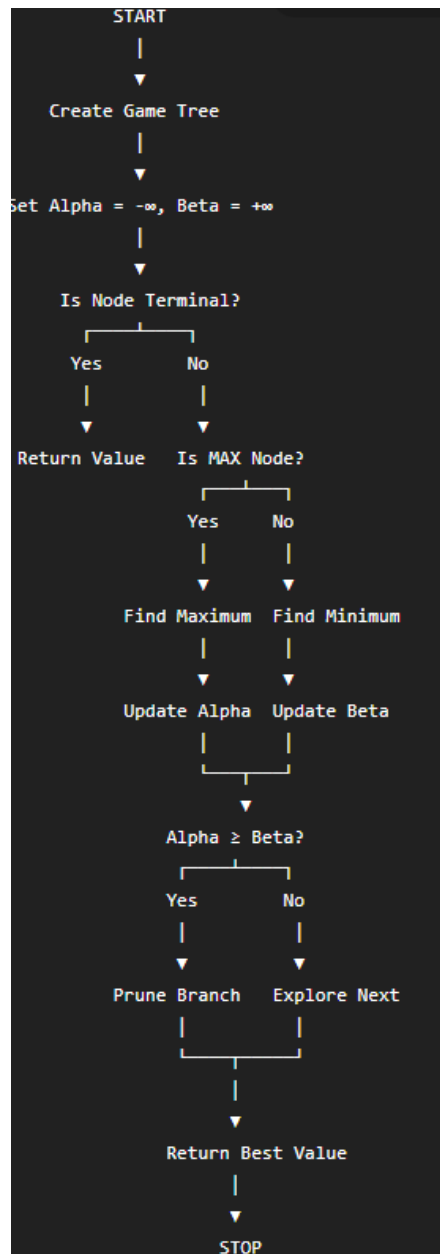
OBJECTIVE

- To understand Alpha-Beta pruning.
- To reduce the number of nodes evaluated in a game tree.
- To improve the efficiency of the Minimax algorithm.
- To find the optimal move for a player.

ALGORITHM

1. Start the program.
2. Create a game tree with leaf node values.
3. Initialize **Alpha** = $-\infty$ and **Beta** = $+\infty$.
4. If the node is a terminal node, return its value.
5. For a MAX node, find the maximum value and update Alpha.
6. For a MIN node, find the minimum value and update Beta.
7. If **Alpha** $\geq$ **Beta**, prune the remaining branches.
8. Return the best value.
9. Display the optimal value.
10. Stop.

FLOWCHART



PYTHON PROGRAM

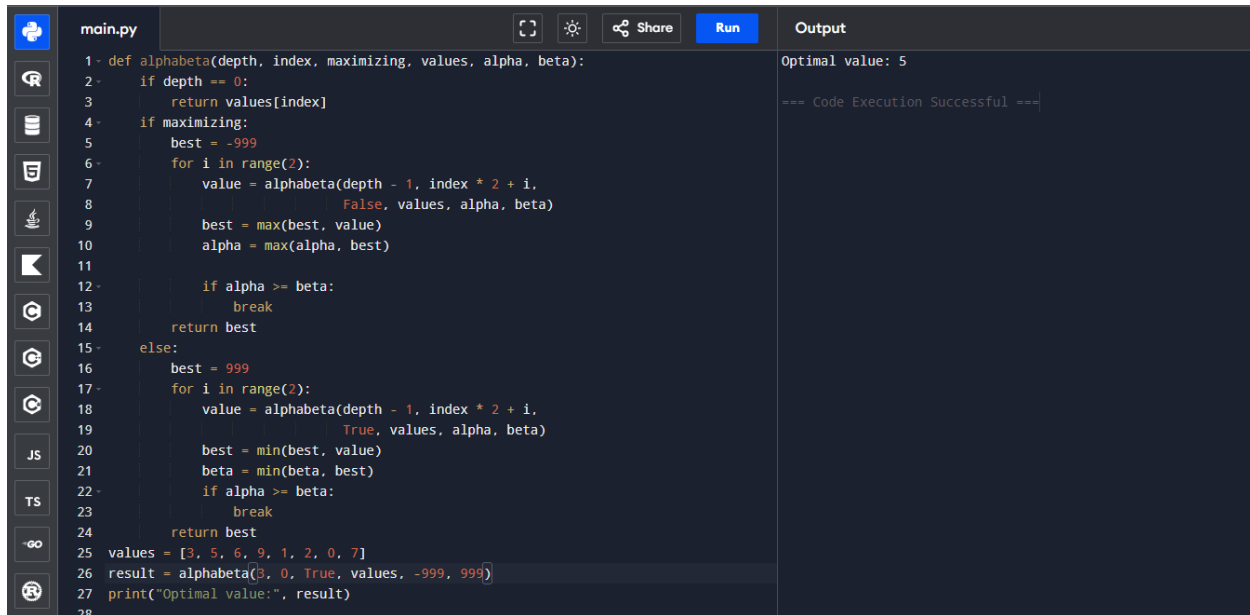
```
def alphabeta(depth, index, maximizing, values, alpha, beta):  
    if depth == 0:  
        return values[index]  
    if maximizing:  
        best = -999  
        for i in range(2):  
            value = alphabeta(depth - 1, index * 2 + i,  
                               False, values, alpha, beta)  
            best = max(best, value)  
            alpha = max(alpha, best)  
            if alpha >= beta:  
                break  
        return best  
    else:  
        best = 999  
        for i in range(2):  
            value = alphabeta(depth - 1, index * 2 + i,  
                               True, values, alpha, beta)  
            best = min(best, value)  
            beta = min(beta, best)  
            if alpha >= beta:  
                break  
        return best
```

```
values = [3, 5, 6, 9, 1, 2, 0, 7]

result = alphabeta(3, 0, True, values, -999, 999)

print("Optimal value:", result)
```

SAMPLE OUTPUT



```
main.py
1 def alphabeta(depth, index, maximizing, values, alpha, beta):
2     if depth == 0:
3         return values[index]
4     if maximizing:
5         best = -999
6         for i in range(2):
7             value = alphabeta(depth - 1, index * 2 + i,
8                               False, values, alpha, beta)
9             best = max(best, value)
10            alpha = max(alpha, best)
11
12            if alpha >= beta:
13                break
14            return best
15     else:
16         best = 999
17         for i in range(2):
18             value = alphabeta(depth - 1, index * 2 + i,
19                               True, values, alpha, beta)
20             best = min(best, value)
21             beta = min(beta, best)
22             if alpha >= beta:
23                 break
24            return best
25 values = [3, 5, 6, 9, 1, 2, 0, 7]
26 result = alphabeta(3, 0, True, values, -999, 999)
27 print("Optimal value:", result)
28
```

Output

Optimal value: 5

=== Code Execution Successful ===

RESULT

Thus, the **Alpha-Beta Pruning algorithm** was successfully implemented in Python, and unnecessary branches of the game tree were pruned.

CONCLUSION

Alpha-Beta pruning improves the efficiency of the Minimax algorithm by avoiding the evaluation of branches that cannot affect the final decision. It helps AI systems make optimal decisions faster in game-playing applications.